

Erreurs

- Qu'est-ce qui peut se passer à l'exécution de ce programme?

```
name = input ( "Provide a file name: " )  
file = open ( name, "r" )  
for line in file:  
    print(line)  
file.close ()
```

Provide a file name:

Erreurs

- Qu'est-ce qui peut se passer à l'exécution de se programme?

```
name = input ( "Provide a file name: " )  
file = open ( name, "r" )  
for line in file:  
    print(line)  
file.close ()
```

```
Provide a file name: nexistepas.txt
```

Erreurs

- Qu'est-ce qui peut se passer à l'exécution de se programme?

```
name = input ( "Provide a file name: " )  
file = open ( name, "r" )  
for line in file:  
    print(line)  
file.close ()
```

```
Provide a file name: nexistepas.txt  
Traceback (most recent call last):  
  File "test.py", line 2, in <module>  
    file = open ( name, "r" )  
FileNotFoundError: [Errno 2] No such  
file or directory: 'nexistepas.txt'
```

Erreur

- Comment d'éviter que le programme plante ?

```
name = input ( "Provide a file name: " )  
try:  
    file = open ( name, "r" )  
    for line in file:  
        print(line)  
    file.close ()  
except:  
    print ( "Error reading file" )
```

Gestion des Exceptions

- Python **lève** une **exception** s'il y a une erreur
- On peut **intercepter** des exceptions en utilisant un bloc `try ... except`

```
name = input ( "Provide a file name: " )  
try:  
    file = open ( name, "r" )  
    for line in file:  
        print(line)  
    file.close ()  
except:  
    print ( "Error reading file" )
```

Gestion des Exceptions

- Il y a beaucoup d'erreurs différentes possibles:
 - Fichier n'existe pas
 - Division par zéro
 - Valeur incorrecte (`int("Bonjour")`)
 - Index incorrect
- Toutes sont interceptées par **except**

Gestion des Exceptions

- Deux erreurs sont possibles ici:

```
num = input ( "Provide a number: " )  
try:  
    print ( 1 / int(num) )  
except:  
    print ( "Error doing calculation" )
```

- Pour toutes les deux, le même message

Gestion des Exceptions

- Interceptor des erreurs spécifiques :

```
num = input ( "Provide a number: " )  
try:  
    print ( 1 / int(num) )  
except ValueError:  
    print ( "No integer provided" )  
except ZeroDivisionError:  
    print ( "Division by zero" )
```

Gestion des Exceptions

- Python montre des messages plus détaillés

```
Provide a file name: nexistepas.txt
Traceback (most recent call last):
  File "test.py", line 2, in <module>
    file = open ( name, "r" )
FileNotFoundError: [Errno 2] No such
file or directory: 'nexistepas.txt'
```

Gestion des Exceptions

- Récupérer le message qui serait montré par Python même

```
num = input ( "Provide a number: " )  
try:  
    print ( 1 / int(num) )  
except ValueError as error:  
    print ( "ValueError: " + str ( error ) )  
except ZeroDivisionError:  
    print ( "Division by zero" )
```

```
Provide a number: age  
ValueError: invalid literal for int()  
with base 10: 'age'
```

Gestion des Exceptions

- Lever des exceptions

```
def askAge ():  
    age = int(input( "Provide your age: " ))  
    if age < 0 or age >= 150:  
        raise ValueError ( "Age out of range" )  
    return age  
  
try:  
    age = askAge ()  
except ValueError as error:  
    print ( "Invalid age: " + str ( error ) )
```

```
Provide your age: 160  
Invalid age: Age out of range
```

Flux d'Exécution

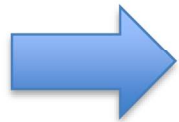


```
def askAge ():
    age = int(input( "Provide your age: " ))
    if age < 0 or age >= 150:
        raise ValueError ( "Age out of range")
    return age

def ask ():
    val = askAge ()
    print ( "Age entered: " + str(val) )
    return val

try:
    age = ask ()
except ValueError as error:
    print ( "Invalid age: " + str ( error ) )
print ( "Program finished" )
```

Flux d'Exécution

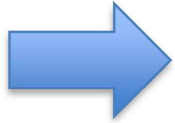


```
def askAge ():
    age = int(input( "Provide your age: " ))
    if age < 0 or age >= 150:
        raise ValueError ( "Age out of range")
    return age

def ask ():
    val = askAge ()
    print ( "Age entered: " + str(val) )
    return val

try:
    age = ask ()
except ValueError as error:
    print ( "Invalid age: " + str ( error ) )
print ( "Program finished" )
```

Flux d'Exécution



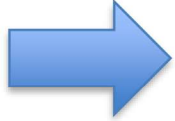
```
def askAge ():
    age = int(input( "Provide your age: " ))
    if age < 0 or age >= 150:
        raise ValueError ( "Age out of range")
    return age

def ask ():
    val = askAge ()
    print ( "Age entered: " + str(val) )
    return val

try:
    age = ask ()
except ValueError as error:
    print ( "Invalid age: " + str ( error ) )
print ( "Program finished" )
```

Provide your age:

Flux d'Exécution



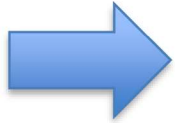
```
def askAge ():
    age = int(input( "Provide your age: " ))
    if age < 0 or age >= 150:
        raise ValueError ( "Age out of range")
    return age

def ask ():
    val = askAge ()
    print ( "Age entered: " + str(val) )
    return val

try:
    age = ask ()
except ValueError as error:
    print ( "Invalid age: " + str ( error ) )
print ( "Program finished" )
```

Provide your age: -2

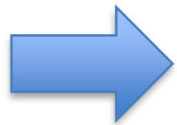
Flux d'Exécution



```
def askAge ():  
    age = int(input( "Provide your age: " ))  
    if age < 0 or age >= 150:  
        raise ValueError ( "Age out of range")  
    return age  
  
def ask ():  
    val = askAge ()  
    print ( "Age entered: " + str(val) )  
    return val  
  
try:  
    age = ask ()  
except ValueError as error:  
    print ( "Invalid age: " + str ( error ) )  
print ( "Program finished" )
```

Provide your age: -2

Flux d'Exécution



```
def askAge ():
    age = int(input( "Provide your age: " ))
    if age < 0 or age >= 150:
        raise ValueError ( "Age out of range")
    return age

def ask ():
    val = askAge ()
    print ( "Age entered: " + str(val) )
    return val

try:
    age = ask ()
except ValueError as error:
    print ( "Invalid age: " + str ( error ) )
print ( "Program finished" )
```

Provide your age: -2

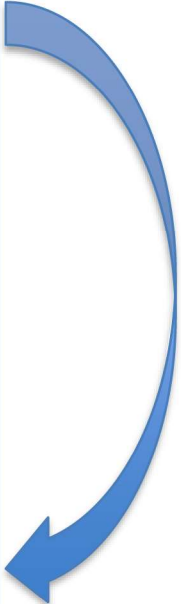
Flux d'Exécution



```
def askAge ():
    age = int(input( "Provide your age: " ))
    if age < 0 or age >= 150:
        raise ValueError ( "Age out of range")
    return age

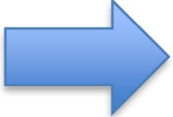
def ask ():
    val = askAge ()
    print ( "Age entered: " + str(val) )
    return val

try:
    age = ask ()
except ValueError as error:
    print ( "Invalid age: " + str ( error ) )
print ( "Program finished" )
```



Provide your age: -2

Flux d'Exécution



```
def askAge ():
    age = int(input( "Provide your age: " ))
    if age < 0 or age >= 150:
        raise ValueError ( "Age out of range")
    return age

def ask ():
    val = askAge ()
    print ( "Age entered: " + str(val) )
    return val

try:
    age = ask ()
except ValueError as error:
    print ( "Invalid age: " + str ( error ) )
print ( "Program finished" )
```

```
Provide your age: -2
Invalid age: Age out of range
```

Flux d'Exécution



```
def askAge ():
    age = int(input( "Provide your age: " ))
    if age < 0 or age >= 150:
        raise ValueError ( "Age out of range")
    return age

def ask ():
    val = askAge ()
    print ( "Age entered: " + str(val) )
    return val

try:
    age = ask ()
except ValueError as error:
    print ( "Invalid age: " + str ( error ) )
print ( "Program finished" )
```

```
Provide your age: -2
Invalid age: Age out of range
Program finished
```